Global routing with FastRoute

What it does Plans which coarse chip regions and metal layers each signal connection should use while trying to avoid routing congestion.

Remember this Start with short routes. Make crowded corridors expensive. Reroute selected connections through less crowded corridors.

A road map for wires

Think of routing as planning roads between houses. The shortest road is attractive, but not if every vehicle needs the same narrow bridge.

OpenROAD's **FastRoute based global router** divides the chip into a coarse grid. Connections cross grid edges with limited routing capacity. A **net** is a set of pins that must be electrically connected. A **via** connects wires on different metal layers.

The important algorithmic steps

1 Build the routing grid. Read placed pins, allowed layers, obstacles and estimated capacity for wires between neighboring regions.

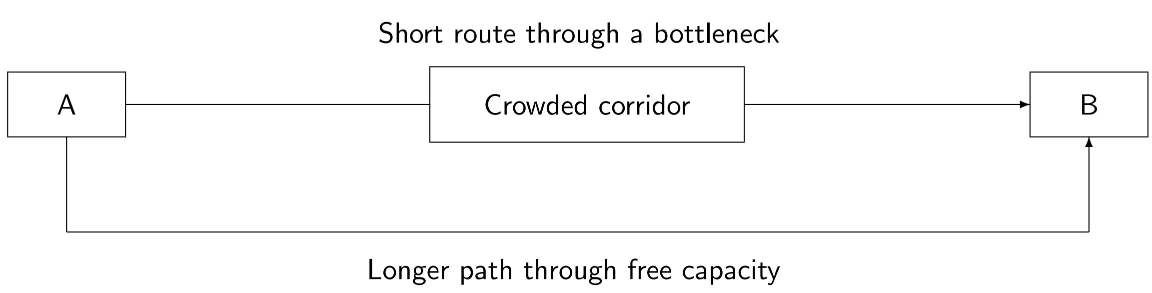
2 Create short connecting trees. Steiner-tree construction can introduce extra branch points to connect a multi-pin net with less total wire. FastRoute uses fast pattern routing for initial connections.

3 Find overloaded corridors. Compare routing demand with capacity. Demand above capacity is congestion overflow.

4 Rip up and reroute. Remove selected route segments. A maze search explores grid neighbors using costs that discourage congestion and excessive wire or vias. Repeated congestion penalties encourage alternate paths.

5 Assign layers and produce guides. Refine the routing plan and hand coarse guides to detailed routing. Detailed routing selects exact tracks and vias and checks physical design rules.

Why the shortest route may lose



This is a conceptual route map. The actual search uses a grid, many interacting nets and several routing layers.

Global routing with FastRoute

One corridor with too many wires

A grid edge has capacity for 5 wire tracks. The current routes need 8. Its overflow is:

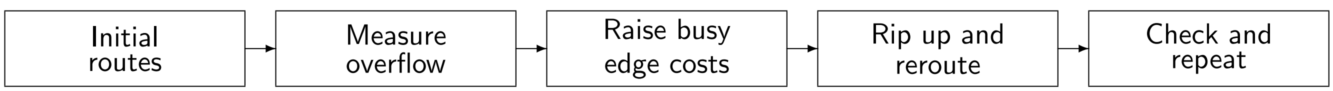
$$\max(0, \text{demand} - \text{capacity}) = \max(0, 8 - 5) = 3$$

If three connections can be moved to another corridor with spare capacity, demand falls to 5 and this edge's overflow becomes zero. The router must also ensure the detour does not simply create a new bottleneck elsewhere.

How maze routing chooses a path

Each grid move has a cost. Search accumulates the costs while exploring possible routes. A path with more grid steps can win if it avoids expensive congested edges.

A teaching example A short route has length cost 4 plus congestion penalty 10: total 14. A detour has length cost 7 and no congestion penalty: total 7. The search prefers the detour. These are invented scores, not FastRoute's exact cost formula.



How routing affects timing closure

Routing determines wire length, layer choices and vias. These change wire resistance and capacitance, which change signal delay. OpenSTA uses updated parasitics to recompute slack; the Resizer can then repair newly visible violations.

Tradeoff A detour may reduce congestion but worsen timing. Timing-critical nets can be prioritized. If the layout is too dense, routing alone may not solve the problem; placement or the floorplan may need revision.

Commands to recognize

```
global_route
```

```
estimate_parasitics -global_routing
```

These require an initialized physical design and valid routing and RC settings. The first produces a global route plan; the second estimates its parasitics for timing analysis. Exact metal geometry is created later by the detailed router, commonly called TritonRoute in OpenROAD.

What to say to your professors

"FastRoute plans connections on a coarse grid. It starts with short routes, detects overused routing corridors, and uses rip-up and maze rerouting to reduce congestion. The output guides detailed routing and provides better wire-delay estimates for timing repair."

The full loop Place cells, build the clock, route wires, analyze timing and repair. Timing analysis and repair repeat across these stages.

Sources and code OpenROAD Global Routing documentation; Pan et al., FastRoute, 2012. Code: `src/grt/src/fastroute/src/FastRoute.cpp`, `maze.cpp`, `maze3D.cpp`. Source review: 7 September 2026; local OpenROAD revision `63fe72c577ef`.